



Pandas Library

SERIES AND DATAFRAMES

To get started with pandas, you will need to get comfortable with its two data structures: **Series** and **DataFrames**.

While they are not a universal solution for every problem, they provide a solid, easy-to-use basis for most applications.

SERIES

A Series is a one-dimensional array-like object containing an array of data (of any NumPy data type) and an associated array of data labels, called its **index**.

The simplest Series is formed from only an array of data:

```
from pandas import Series

seriesobj = Series([1, 2, -3, 4.1])

print(seriesobj)
```

Default indexes Values

0 1.0

1 2.0

2 -3.0

3 4.1

dtype: float64

Since we did not specify an index for the data, a **default** one consisting of the integers 0 through N - 1 (where N is the length of the data) is created.

You can get the array representation and index object of the Series via its values and index attributes, respectively:

```
print(seriesobj.values)
print(seriesobj.index)
```

You can create series with an index in order to identify each data point.

```
seriesobj2 = Series([1, 4, 5, 9], index=['a', 'b', 'c', 'd'])  
print(seriesobj2)
```

Output:

```
a      1  
b      4  
c      5  
d      9  
dtype: int64
```


You can create a Series from a dict in Python.

```
data = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}  
seriesdata = Series(data)  
print(seriesdata)
```

SERIES

```
data = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}
```

```
states = ['California', 'Ohio', 'Oregon', 'Texas']  
obj = Series(data, index=states)  
print(obj)
```

OUTPUT:

```
California    NaN  
Ohio          35000.0  
Oregon        16000.0  
Texas         71000.0  
dtype: float64
```

SERIES

Values found in data were placed in the appropriate locations, but since no value for 'California' was found, it appears as **NaN** (not a number) which is considered in pandas to mark missing or NA values. The *isnull* and *notnull* functions in pandas should be used to detect missing data.

```
import pandas as pd # this needs to go
                        # at the beginning of the file
print(pd.isnull(obj))
print(pd.notnull(obj))
```

OUTPUT:

```
California    True ←
Ohio          False
Oregon        False
Texas         False
dtype: bool
California    False ←
Ohio          True
Oregon        True
Texas         True
dtype: bool
```


DATAFRAME

A DataFrame represents a tabular, spreadsheet-like data structure containing an ordered collection of columns, each of which can be a different value type (numeric, string, boolean, etc.).

The DataFrame has both a row and column index;

DataFrame is a data structure (just like list and dictionary are data structures), included in the pandas package.

Create a simple DataFrame

Syntax

`pandas.DataFrame(data, index, columns, dtype, copy)`

Parameter	Type	Default	Description
data	ndarray, series, maps, lists, dictionaries, constants, or another DataFrame	None	
index	index or array-like	range (0 - n)	Indexes are used as row labels
column	index or array-like	range (0 - n)	These are column labels, or column headers
dtype	data tye	inferred	Used to force a single data type
copy	boolean	FALSE	Copy data from inputs. Only affects DataFrame / 2d ndarray input

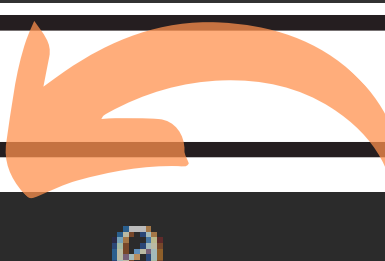
Define the first parameter, data, as a simple list
['A', 'B', 'C']

```
import pandas as pd

dataframe1 = pd.DataFrame(['A', 'B', 'C'])

print(dataframe1)
```

OUTPUT



0	
0	A
1	B
2	C

Notice the indexes (or, row labels), on the left.
They default to a range sequence starting with 0.

CREATE A DATAFRAME WITH A LIST ARRAY

Practice using parameters by passing both index and column labels, and force the data to be of type float.

```
pandas.DataFrame(data, index, columns, dtype, copy)
```

```
dataframe2 = pd.DataFrame(  
    [  
        [10, 2, 3],  
        [4, 5, 6],  
        [7, 8, 9],  
        [98, 99, 100]  
    ],  
    index=['Student 1', 'Student 2', 'Student 3', 'Student 4'],  
    columns=['Course 1', 'Course 2', 'Course 3'],  
    dtype=float  
)
```

OUTPUT:

	Course 1	Course 2	Course 3
Student 1	10.0	2.0	3.0
Student 2	4.0	5.0	6.0
Student 3	7.0	8.0	9.0
Student 4	98.0	99.0	100.0

CREATE A DATAFRAME WITH A DICTIONARY OF LISTS

```
dataframe3 = pd.DataFrame(  
    {'Id': [1, 2, 3],  
     'ClientType': [0, 1, 1],  
     'Name': ['Andrew, Johnson',  
              'Sue, Anderson',  
              'Roberta, Leins'],  
     'Sex': ['male', 'female', 'female'],  
     'Age': [22.0, 38.0, 26.0]  
    })  
print(dataframe3)
```

OUTPUT:

	Id	ClientType	Name	Sex	Age
0	1	0	Andrew, Johnson	male	22.0
1	2	1	Sue, Anderson	female	38.0
2	3	1	Roberta, Leins	female	26.0

COPY A DATA STRUCTURE TO A DATAFRAME

Another way of creating a DataFrame is to copy from an existing data structure such as: a list of dictionaries, narrays/lists, Series (a pandas structure), and a list of Series.

```
dict = {"country": ["Brazil", "Russia", "India", "China", "South Africa"],
        "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
        "area": [8.516, 17.10, 3.286, 9.597, 1.221],
        "population": [200.4, 143.5, 1252, 1357, 52.98]}

print(pd.DataFrame(dict))

#or
print(pd.DataFrame.from_dict(dict))
```

Select specific columns

There are three methods of selecting columns from a DataFrame, and they all return the same results:

- dot notation:

`dataframe.column_name`

- square braces with the name of the column as a string:

`dataframe['column_name']`

- numeric indexing and the iloc selector :

`dataframe.iloc[:<column_number>]`

```
dict = {"country": ["Brazil", "Russia", "India", "China", "South Africa"],
        "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
        "area": [8.516, 17.10, 3.286, 9.597, 1.221],
        "population": [200.4, 143.5, 1252, 1357, 52.98]}

dataframe4 = pd.DataFrame(dict)
print(dataframe4.country)
print(dataframe4['country'])
print(dataframe4.iloc[:, 1])
```


(include **head()** method to limit rows returned)

DataFrame - head() function

The head() function is used to get the first n rows.

This function returns the first n rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it.

Syntax:

```
DataFrame.head(self, n=5)
```

Viewing the first 5 lines

```
df.head()
```

```
print(dataframe4.country.head())  
print(dataframe4['country'].head())  
print(dataframe4.iloc[:, 1].head())
```

Select multiple columns

Include multiple column names as a list.

```
print(dataframe4[['country', 'population']].head())
```

	country	population
0	Brazil	200.40
1	Russia	143.50
2	India	1252.00
3	China	1357.00
4	South Africa	52.98

Select specific rows

(I) Select the first set of 3 rows by slicing inside of brackets.

```
print(dataframe4[0:3])  
  
print(dataframe4.head(3))
```

(II) Select the third and fourth row from the DataFrame.

```
print(dataframe4[2:4])
```

OUTPUT:

	country	capital	area	population
2	India	New Delhi	3.286	1252.0
3	China	Beijing	9.597	1357.0

Notice how the index (left side) is the index for each specific record in the DataFrame, not for the selection result's table row.

Select specific columns and rows at the same time

Columns get listed first, then rows are sliced.

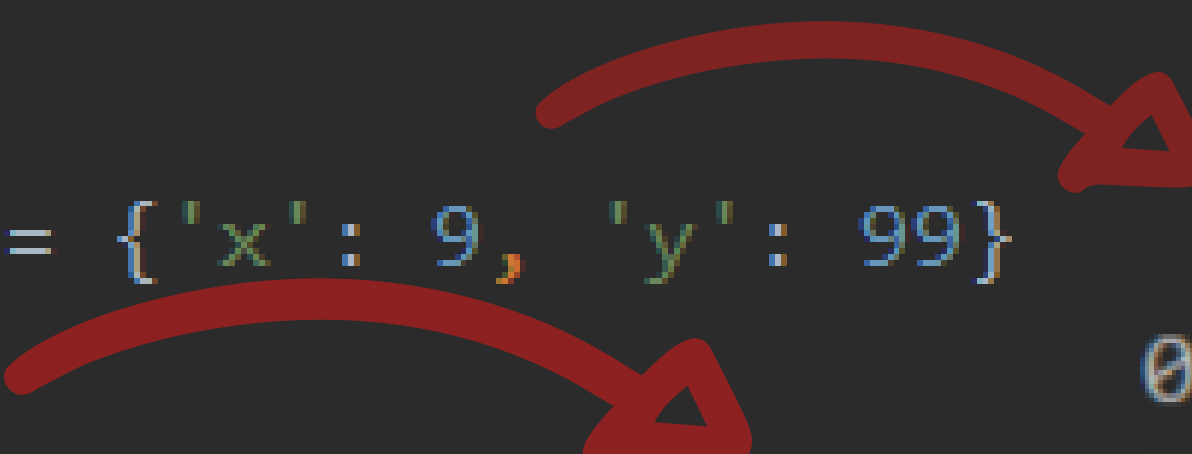
```
print(dataframe4[['country', 'population']][3:5])  
# or  
print(dataframe4[['country', 'population']][len(dict)-1:len(dict)+1])
```

OUTPUT:

3	China	1357.00
4	South Africa	52.98

You can also assign a dict to a row of a DataFrame:

```
x = pd.DataFrame({'x': [1, 2, 3], 'y': [3, 4, 5]})  
print(x)  
x.iloc[1] = {'x': 9, 'y': 99}  
print(x)
```



	x	y
0	1	3
1	9	99
2	3	5

EXERCISES

- (I) Define the products dict and then pass it to pandas DataFrame constructor:

```
products = {  
    'Product O': [100, 200, 0, 100],  
    'Product T': [0, 300, 700, 200],  
    'Product Y': [0, 880, 1400, 1010]  
}  
  
orders = pd.DataFrame(products)  
  
print(orders)
```

- Each (key, value) item in data corresponds to a column in the resulting DataFrame.
- The Index of this DataFrame was added on creation as the numbers 0-3 (default), but we could also create our own when we initialize the DataFrame.

'Product O': [100, 200, 0, 100],

EXERCISES

	Product O	Product T	Product Y
0	100	0	0
1	200	300	880
2	0	700	1400
3	100	200	1010

Default indexes

(II) Add customer names as indexes:

```
orders = pd.DataFrame(products, index=['John', 'Joe', 'Rob', 'Mary'])
```

	Product O	Product T	Product Y
John	100	0	0
Joe	200	300	880
Rob	0	700	1400
Mary	100	200	1010

EXERCISES

(III) **Locate** a customer using his name:

```
productsBoughtByJoe = orders.loc['Joe']  
print(productsBoughtByJoe)
```

```
Product 0      200  
Product T      300  
Product Y      880  
Name: Joe, dtype: int64
```


LOC AND ILOC

WHAT'S THE DIFFERENCE?

loc in Pandas

loc is label-based, which means that we have to specify the **name** of the rows and columns that we need to filter out.

iloc in Pandas

On the other hand, iloc is integer index-based. So here, we have to specify rows and columns by their **integer index**.

EXERCISES

(IV) Create the following sample DataFrame:

```
      name  quantity  unit_price  section  producer
0    Pencils    1000      2.35      A      X
1  Notebooks    1020     10.20      B      T
2   Erasers     300      1.20      C      L
3 Pencil Sharpeners    120      0.90      A      L
4   Planner    4500     50.25      B      Z
```

EXERCISES

(IV) Create the following sample DataFrame:

```
stocks = {  
    'name': ['Pencils', 'Notebooks', 'Erasers', 'Pencil Sharpeners', 'Planner'],  
    'quantity': [1000, 1020, 300, 120, 4500],  
    'unit_price': [2.35, 10.2, 1.2, 0.9, 50.25],  
    'section': ['A', 'B', 'C', 'A', 'B'],  
    'producer': ['X', 'T', 'L', 'L', 'Z']  
}  
  
stocksd = pd.DataFrame(stocks)
```

EXERCISES

Find all the rows based on any condition in a column

One thing we use almost always when we're exploring a dataset – filtering the data based on a given condition.

For example, we might need to find all the rows in our dataset where quantity is more than x pieces, or the section is 'A', and so on.

We can solve types of queries with a simple line of code using *pandas.DataFrame.loc[]*.

We just need to pass the condition within the loc statement.

EXERCISES

(V) Find the rows where the value of quantity is greater than or equal to 1000:

```
print(stocksdf.loc[stocksdf.quantity >= 1000])
```

	name	quantity	unit_price	section	producer
0	Pencils	1000	2.35	A	X
1	Notebooks	1020	10.20	B	T
4	Planner	4500	50.25	B	Z

(VI) Similarly, we can also use multiple conditions to filter our data, such as finding all the rows where the quantity is greater than or equal to 1000 and the section is 'B':

```
print(stocksdf.loc[(stocksdf.quantity >= 1000) & (stocksdf.section == 'B')])
```

	name	quantity	unit_price	section	producer
1	Notebooks	1020	10.20	B	T
4	Planner	4500	50.25	B	Z

EXERCISES

Select a range of rows using loc

Using loc, we can also slice the Pandas dataframe over a range of indices.

If the indices are not in the sorted order, it will select only the rows with index 1 and 3.

```
print(stocksdf.loc[1:3])
```



	name	quantity	unit_price	section	producer
1	Notebooks	1020	10.2	B	T
2	Erasers	300	1.2	C	L
3	Pencil Sharpeners	120	0.9	A	L

EXERCISES

And if the indices are not numbers, then we cannot slice our dataframe.
In that case, we need to use the `iloc` function to slice our Pandas dataframe.

```
stocks = {  
    'name': ['Pencils', 'Notebooks', 'Erasers', 'Pencil Sharpeners', 'Planner'],  
    'quantity': [1000, 1020, 300, 120, 4500],  
    'unit_price': [2.35, 10.2, 1.2, 0.9, 50.25],  
    'section': ['A', 'B', 'C', 'A', 'B'],  
    'producer': ['X', 'T', 'L', 'L', 'Z']  
}  
stocksdf = pd.DataFrame(stocks, index=['a', 'b', 'c', 'd', 'e'])
```

Alpha-
numeric
index

```
print(stocksdf.loc[1:3])
```

→ **error**

***Remember to remove the indexes added for this example from stocksdf constructor

EXERCISES

Select only required columns with a condition

We can also select the columns that are required of the rows that satisfy our condition.

For example, if our dataset contains hundreds of columns and we want to view only a few of them, then we can add a list of columns after the condition within the loc statement itself:

```
print(stocksdf.loc[(stocksdf.unit_price >= 10), ['unit_price', 'producer']])
```

	unit_price	producer
1	10.20	T
4	50.25	Z



EXERCISES

Update the values of a **particular** column on selected rows

We can do this by running a for loop as well but if our dataset is big in size, then it would take forever to complete the task. Using loc in Pandas, we can do this within seconds, even on bigger datasets!

We often have to update values in our dataset based on a certain condition.

For example, if the price values are greater than equal to 10, then we want to update the values of the column section to be "W"

```
stocksdf.loc[(stocksdf.unit_price >= 10), ['section']] = 'W'
```

	name	quantity	unit_price	section	producer
0	Pencils	1000	2.35	A	X
1	Notebooks	1020	10.20	W	T
2	Erasers	300	1.20	C	L
3	Pencil Sharpeners	120	0.90	A	L
4	Planner	4500	50.25	W	Z



EXERCISES

Update the values of **multiple** columns on selected rows

```
stocksdf.loc[(stocksdf.section == 'A'), ['unit_price', 'quantity']] = [20.00, 1000]
```

	name	quantity	unit_price	section	producer
0	Pencils	1000	20.00	A	X
1	Notebooks	1020	10.20	W	T
2	Erasers	300	1.20	C	L
3	Pencil Sharpeners	1000	20.00	A	L
4	Planner	4500	50.25	W	Z



EXERCISES

Select rows with indices using iloc.

When we are using iloc, we need to specify the rows and columns by their integer index.

If we want to select only the first and third row, we simply need to put this into a list in the iloc statement with our dataframe:

```
print(stocksdf.iloc[[0,2]])
```

	name	quantity	unit_price	section	producer
0	Pencils	1000	20.0	A	X
2	Erasers	300	1.2	C	L



EXERCISES

Select rows with particular indices and particular columns

Earlier, we selected a few columns from the dataset using the loc function.

We can do this using the iloc function.

Keep in mind that we need to provide the **index** number of the column instead of the column name:

```
print(stocksdf.iloc[[0, 2], [1, 3]])
```

	quantity	section
0	1000	A
2	300	C



EXERCISES

Select a range of rows using iloc

We can slice a dataframe using iloc as well.

We need to provide the start_index and end_index+1 to slice a given dataframe.

If the indices are not the sorted numbers even then it will select the starting_index row number up to the end_index:

```
print(stocksdf.iloc[1:3])
```

	name	quantity	unit_price	section	producer
1	Notebooks	1020	10.2	W	T
2	Erasers	300	1.2	C	L



EXERCISES

Select a range of rows and columns using `iloc`

Slice the data frame over both rows and columns. In the below example, we selected the rows from (1-2) and columns from (2-3).

```
print(stocksdf.iloc[1:3, 2:4])
```

	name	quantity	unit_price	section	producer
1	Notebooks	1020	10.2	W	T
2	Erasers	300	1.2	C	L

